

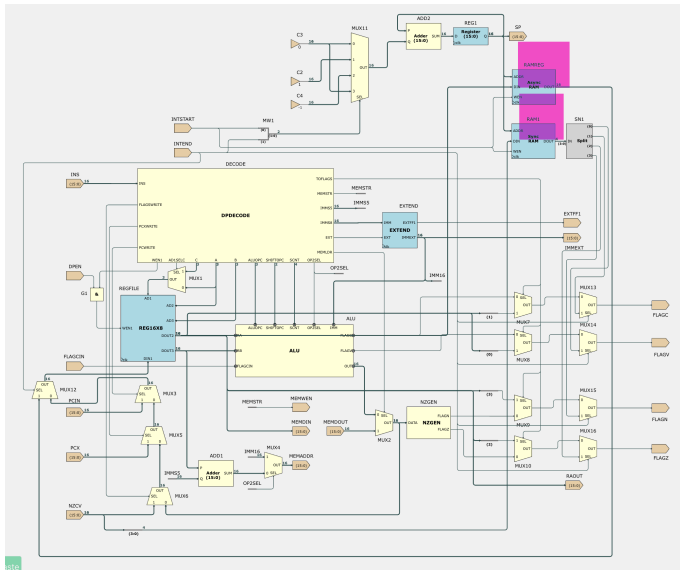
Section 3: Challenge

2026年3月19日 星期四 08:40

The interrupt mechanism in EEP11 is deliberately minimal: it provides just enough hardware to allow software to save and restore the entire CPU state during an interrupt. As you have seen in this lab, this results in relatively complex and time-consuming interrupt software.

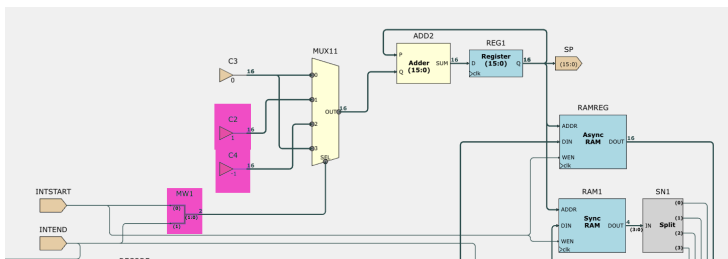
For this challenge you will design an extended version of EEP1, called EEP1S, that adds specialised hardware support for re-entrant interrupts using a stack.

This is my version of EEP1S. Compared to software stack in previous section, I implemented individual hardware to store/load Flags & register automatically, which uses 2 Async RAMs separately.



I made my design mainly in DATAPATH. When there is an interrupt happening (both nested and non-nested are fine), INTSTART and INTEND (from CONTROLPATH) will be set high at the beginning and in the end, so MW1 will be a 2-bit signal to help us push/pop in the stack:

- 00: Nothing happens
- 01: Stack pushing
- 10: Stack popping
- 11: This can never happen, so we just connect to some meaningless thing.

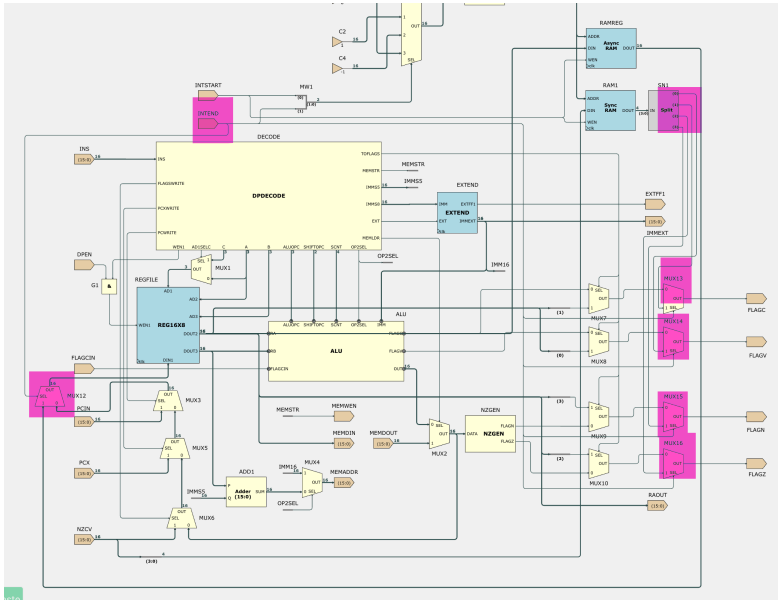


So each time pushing, we increment address in both RAMs by 1 and store Flags and register value into RAMs automatically.

Each time popping, we decrement address by 1 and load values.

Note that in normal stack, we start from large address like 0xEEFF in main RAM to build stack. In my case I start from default value 0x0000 as I use individual RAMs





For loading, we add MUX12 to control writing register, and MUX13; 14; 15; 16 to control writing NZCV. Note that loading needs 1 cycle and WEN = 1, so we have to write some meaningless MOV delays in software.

When INTEND = 1, we will pop these values.

```

nested_interrupts.txt
SETI

LOOP  ADD R0, #1
      JNE LOOP

      ORG 0xFF00

ISR    LDR R0, [0xFFFE]    // read interrupt status
      AND R0, #2          // test IRQ1
      JNE ISR1            // if IRQ1 active, go to ISR1

ISR0   MOV R0, #1          // IRQ0 reset high
      STR R0, [0xFFFF]
      MOV R0, #0          // IRQ0 reset low
      STR R0, [0xFFFF]

      SETI                // open nesting window
      ADD R3, #1          // payload for IRQ0
      MOV R0, #3          // optional delay in previous tests
      SUB R0, #1
      JNE -1

      CLRI                // close nesting window before return
      RETINT

ISR1   MOV R0, #2          // IRQ1 reset high
      STR R0, [0xFFFF]
      MOV R0, #0          // IRQ1 reset low
      STR R0, [0xFFFF]

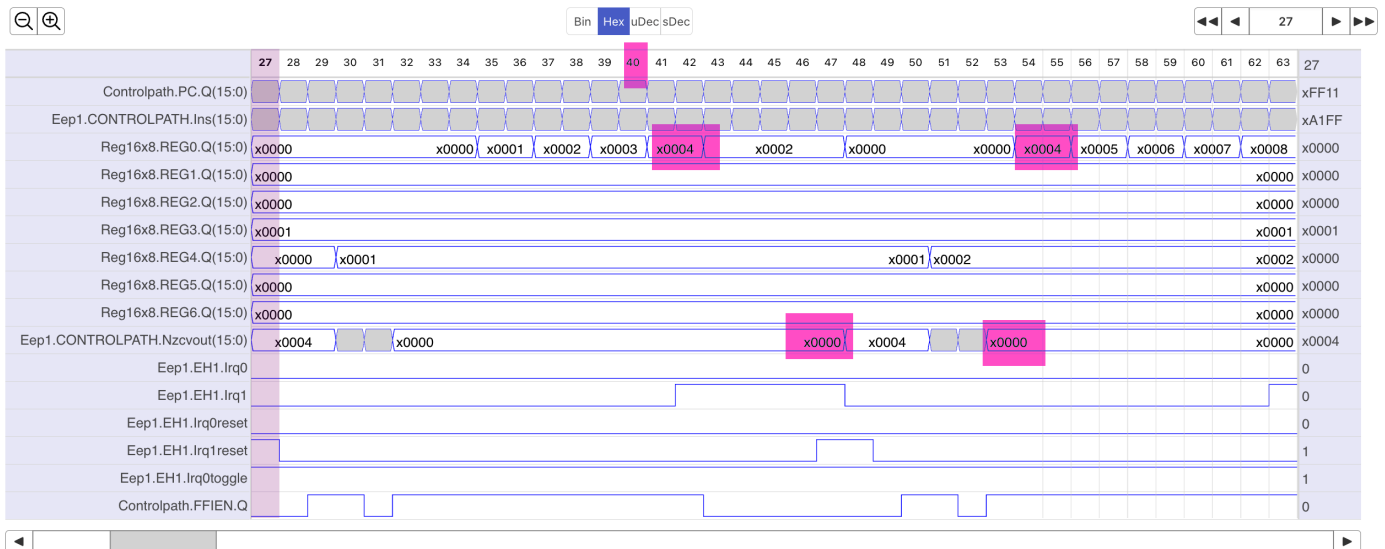
      SETI
      ADD R4, #1          // payload for IRQ1

      CLRI                // close nesting window before return
      RETINT
  
```

Compared to the software design, hardware one has the following advantages:

1. Simplify the software assembly language, we don't have to write store/load lines.
2. R5 and R6 can be used now, but in software one they're only used to pass values.
3. Can do nested interrupts mutually, which is more suitable for real life cases.

While it has disadvantage of more components used, so costs more.



From the test, we can see that R0's value (x0004) and NZCV (0x0000) preserved correctly, so it worked.